IB Computer Science HL Internal Assessment

“TherapySort: A therapist sorting desktop application”

Criterion: A, B, C, E

Candidate Code: lyy607

Table of Contents:

Criterion A – Planning

Defining the Problem

My client, Mr. Baseer Rajabov, is a Year 13 student at an IB school in London. The IB is an incredibly demanding and academically challenging programme, and as Mr. Rajabov is in his final year, he has found the programme even more demanding. He has decided to put emphasis on prioritizing his mental health in order to manage himself effectively and healthily. He primarily uses online therapists, as they are more convenient and can be worked into his schedule more easily without the need to commute to an office. However, browsing through websites for hours in order to find the right therapist for him has proven to be a taxing and time-consuming process, as he often needs to go through a large number of different therapist profiles.

As such, we had an initial meeting, where Mr. Rajabov requested that I build him an application that would automatically sort therapists from his preferred website by a series of criteria that he himself would provide (see Appendix 1 for full transcript). Through email correspondence after the fact, Mr Rajabov requested certain parameters that he would like considered, qualifications, job titles, location, keywords, verification status, endorsements, and availability. He also stated in a follow-up email that a GUI was not entirely necessary, but would be overall preferred (Appendix 2).

[Words: 206]

Rationale for Solution

As a solution, I have proposed a Therapist Sorting desktop application for Mr. Rajabov that will include all features he requested.

Mr. Rajabov wanted therapist information sorted from a specific website, which then provides him with an interface that allows him to sort through certain parameters. However, scraping data from the chosen website would be considered a violation of its terms of service. This was communicated to him, and he agreed that the system could instead be developed using mock data with the same parameters he specified (Appendix 3). This still meets his needs as a client because the primary goal of the system is to demonstrate filtering and sorting functionality. If required in the future,

the mock dataset could easily be replaced with a live dataset with the same structure.

Whilst he stated that a GUI is preferred but not necessary, I will treat the GUI as a necessary component of this application, as the system is intended for use by a non-technical user. A GUI with dropdown filters and search fields allows faster and easier navigation than a command-line interface.

The frontend will be implemented using Unity (C#), as the therapist profiles can be generated dynamically using reusable UI prefabs. This allows the system to display any number of therapists returned by a search query. The backend will use Python (pandas, Flask, and SQLite) to process CSV data, store therapist records in a structured database, and query the dataset when filters are applied.

[Words: 245]

Criteria for Success

- The system successfully imports therapist profile data (names, phone numbers, qualifications, job titles, location, description, verification status, and availability) from a CSV dataset into the application database
- The user can filter therapist profiles by qualifications, job titles, location, keywords (in the description), verification status, endorsements, and availability
- The system allows multiple filters to be applied simultaneously (eg. location AND qualification AND availability)
- Therapist profiles are displayed in a graphical user interface with clear headings and interactive filter controls
- The user can specify the number of therapist profiles displayed in the search results
- The system displays clear error messages when invalid input is entered or when no therapists match the selected filters
- Search results should be generated within 5 seconds after filters are applied

Criterion B – Solution Overview

TESTING PLAN

Success criteria no.	Test description	Input data	Expected result
1	Import therapist profile data from the CSV dataset when the application loads	Load/open application	Be able to access all database information
2	Filter therapist profiles using search parameters	Add additional parameters to filter by	Display information adhering to the filters
3	Apply multiple search filters simultaneously	Apply filters such as location = "London" and qualification = "MA"	Only therapist profiles matching both filters are displayed
4	Display therapist profiles in the graphical user interface	Open application	Therapist profiles are displayed in the GUI with clear headings and filter controls
5	Specify the number of therapist profiles displayed in search results	Select maximum number of therapists = 5	Only 5 therapist profiles are displays
6	Display error message when invalid input or empty search results occur	Leave required search field empty or enter invalid data	Error message displayed (eg. “Please select a location from the dropdown menu”)
7	Measure search result generation time after filters are applied	Apply filters such as location = "London" and qualification = "MA"	Search results are displayed within 5 seconds

UML Class Diagram

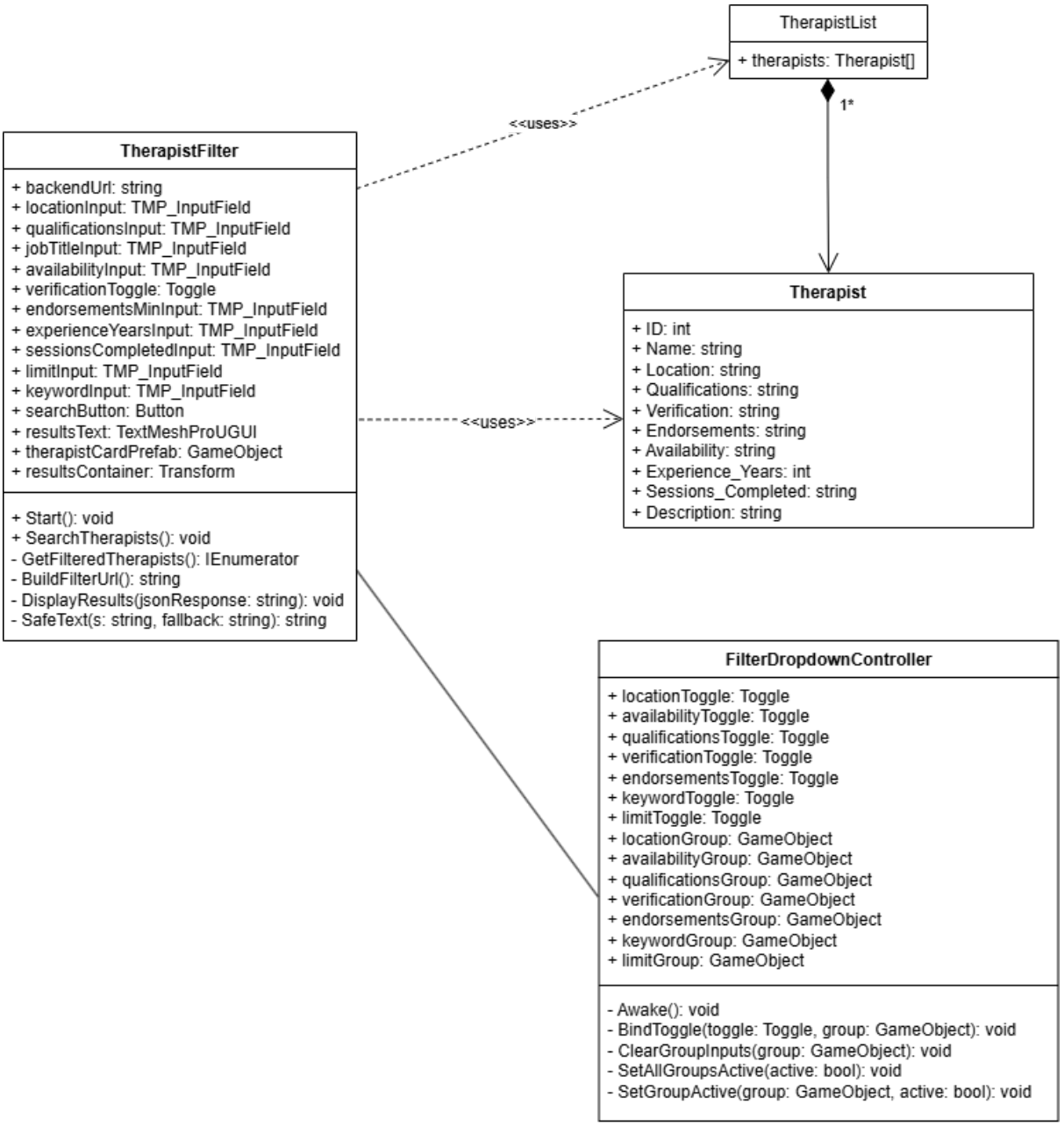


Figure 1: UML Class Diagram showcasing relationship between different C# scripts.

UML Use Case Diagram

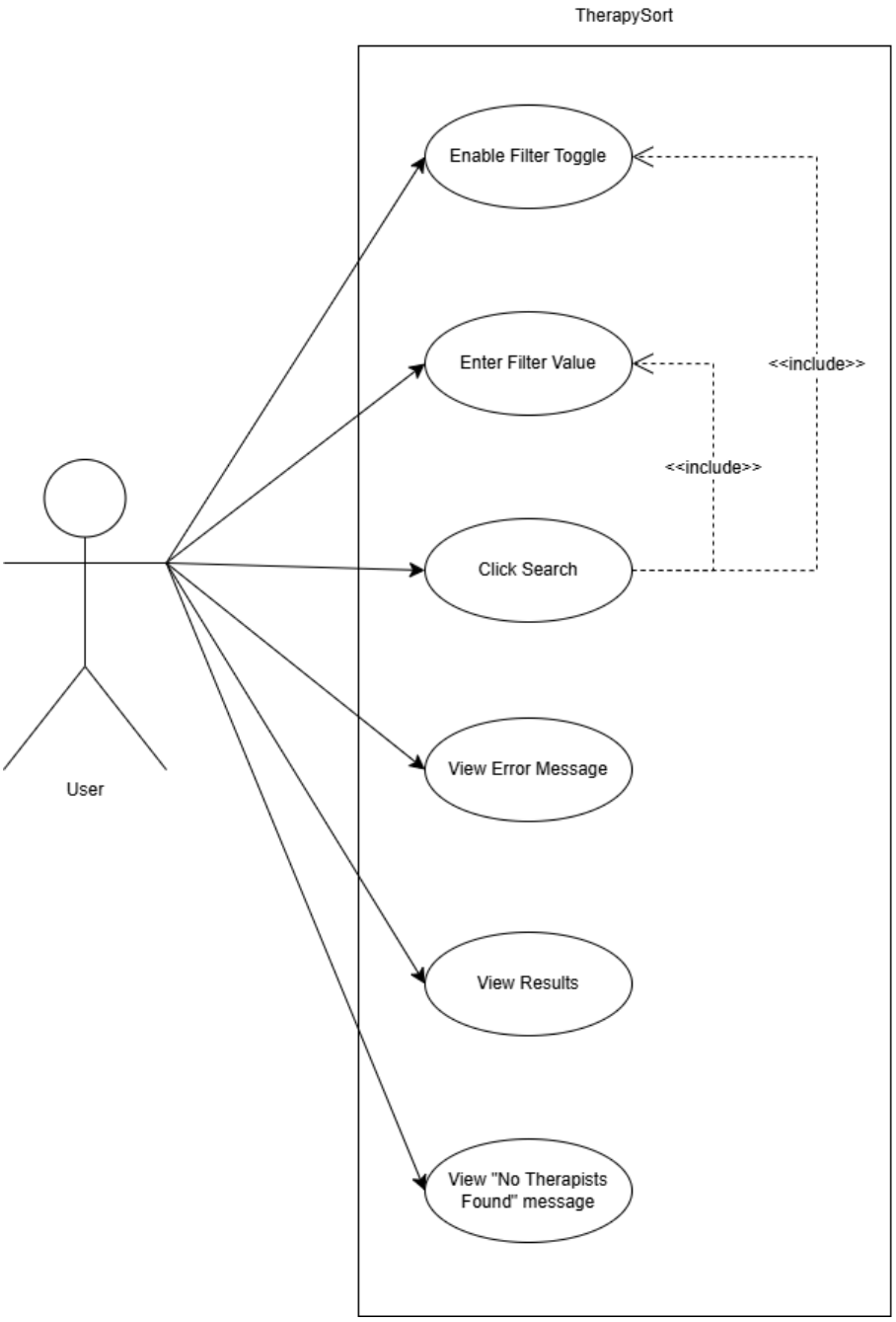


Figure 2: Use case diagram showcasing use case relations for the user within TherapySort

UML Sequence Diagram

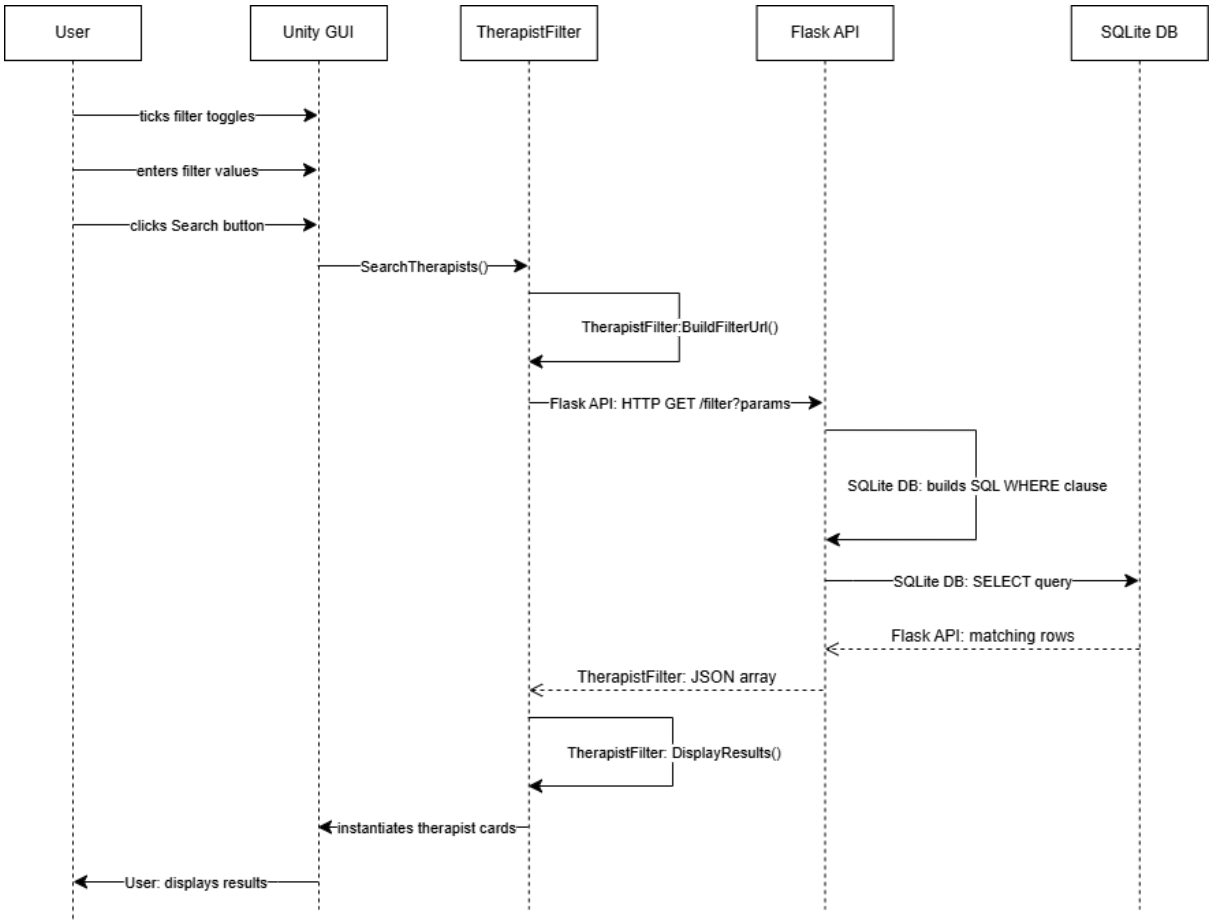


Figure 3: Sequence diagram for the user to SQLite database request cycle.

Single Entity Relationship Diagram

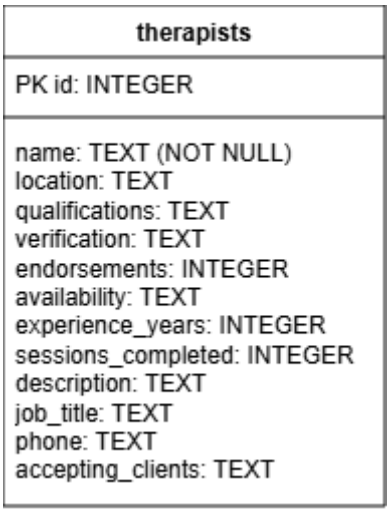


Figure 4: Single entity relationship diagram. Given that therapists is a standalone table, no other relationships are displayed.

Data Dictionary

Field	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique identifier for each therapist
name	TEXT	NOT NULL	Full name of the therapist
location	TEXT	N/A	City/country or “Online Only”
qualifications	TEXT	N/A	Academic/professional qualifications
verification	TEXT	N/A	Verification status
endorsements	INTEGER	N/A	Number of endorsements received
availability	TEXT	N/A	Days of the week/times available
experience_years	INTEGER	N/A	Years of professional experience
sessions_completed	INTEGER	N/A	Total sessions completed
description	TEXT	N/A	Therapist’s self-description
job_title	TEXT	N/A	Professional role/title
phone	TEXT	N/A	Contact phone number
accepting_clients	TEXT	N/A	Accepting / Not Accepting / Waitlist

CSV to SQLite Import Algorithm Flowchart

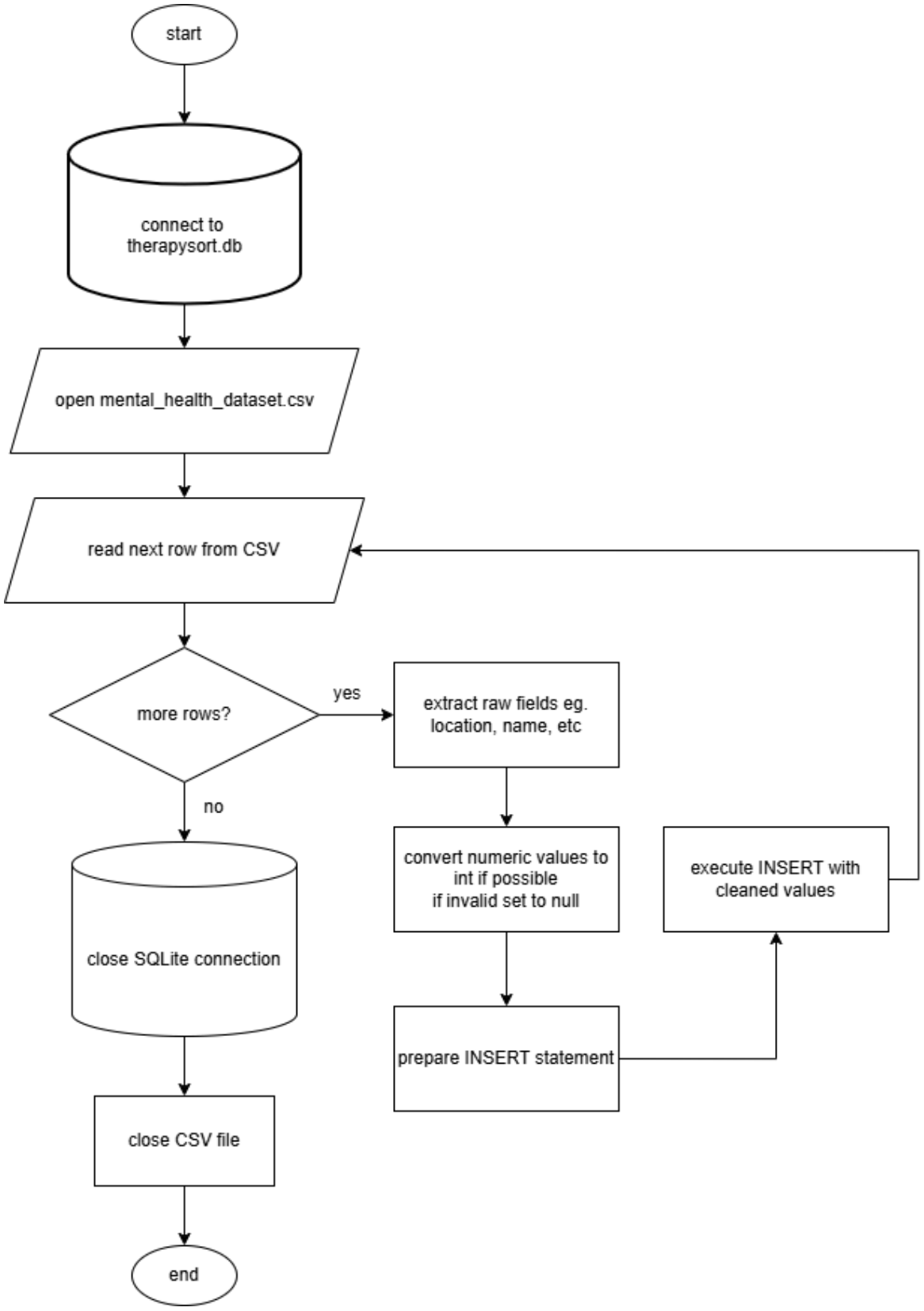


Figure 5: CSV to SQLite import algorithm flowchart.

Backend Flowchart

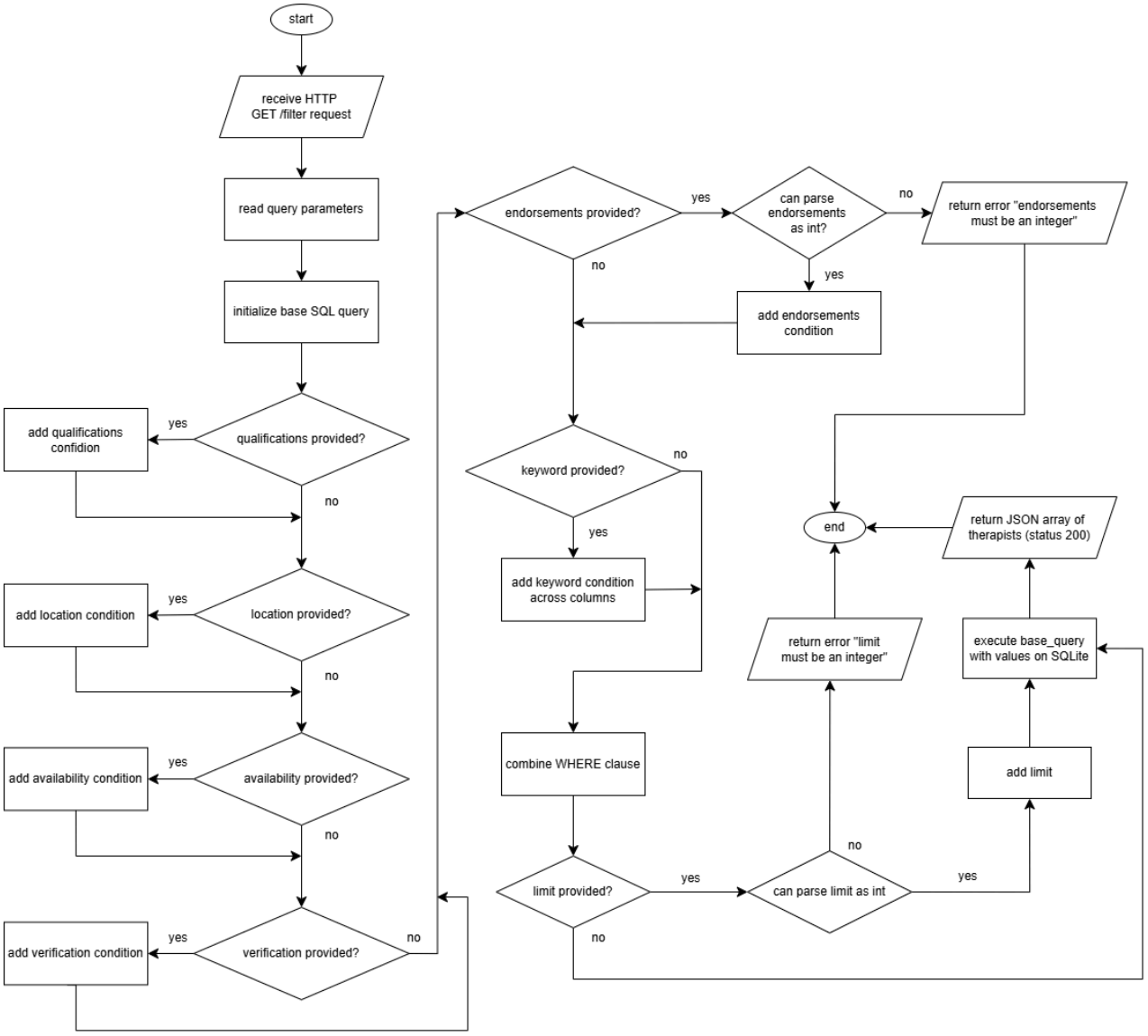


Figure 6: Flowchart for backend.py

Unity Search + Card Rendering Flowchart

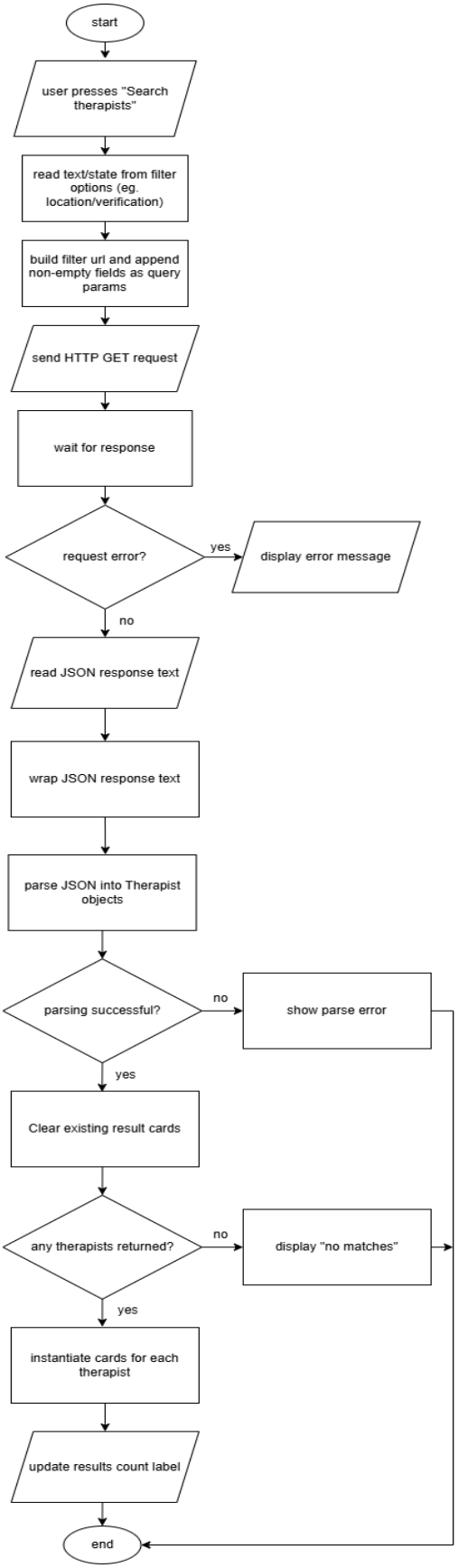


Figure 7: Unity search and card rendering algorithm flowchart.

Initial GUI Designs

Figure 8: Preliminary design for the TherapySort GUI.

Figure 9: Initial Hierarchy for the TherapySort GUI Unity scene.

Record of Tasks

Task Number	Planned action	Planned outcome	Time estimate	Target completion date	Criterion
1	Initial meeting with client to discuss the problem	Understand the client's needs and requirements for a therapist sorting application	1h	**07/07/2025**	**A**
2	Follow-up email with client to confirm desired features and filtering parameters	Confirm system requirements including filtering criteria, data fields, and preferred GUI interface	0.5h	10/07/2025	**A**
3	Write Criterion A documentation (scenario, rationale, success criteria)	Completed planning documentation submitted to teacher	2h	14/07/2025	**A**
4	Create system architecture and UML diagrams (class, use case, sequence, ERD)	Diagrams showing interaction between Unity frontend, Python backend, and SQLite database	3h	21/07/2025	**B**
5	Create flowcharts for CSV import algorithm, backend filter logic, and Unity card rendering	Visual representation of how key algorithms function in the system	2h	23/07/2025	**B**
6	Create data dictionary and design initial GUI mockups	Data dictionary defining all database fields; annotated screen mockups for the Unity interface	1.5h	25/07/2025	**B**
7	Write test plan aligned to success criteria	Test plan table covering all 7 success criteria with expected results	1h	28/07/2025	**B**
8	Source and modify mock therapist CSV dataset to include all required filter fields	Mock dataset of 1000 therapist records with all required parameters present	1h	04/08/2025	**C**
9	Develop init_db.py to initialise SQLite database schema	therapySort.db created with correct table structure and constraints	2h	09/08/2025	**C**
10	Develop importCsvSqlite.py to import CSV data into SQLite database	All 1000 therapist records successfully inserted into the database	0.5h	09/08/2025	**C**
11	Implement filtering queries in backend.py Flask server	Backend returns correctly filtered therapist profiles as JSON based on active parameters	3h	12/08/2025	**C**
12	Test Unity connection with Flask backend using HTTP_Test.cs	Unity successfully sends GET requests and receives JSON responses from Flask	1h	19/08/2025	**C**
13	Develop Unity GUI layout and therapist card prefab system	Interface displaying therapist profiles dynamically as reusable prefab cards	2h	22/08/2025	**C**
14	Implement	Filter toggles correctly	2h	26/08/2025	**C**

	FilterDropdownController.cs with toggle and group binding	show/hide input groups and clear inputs on deactivation			
15	Connect Unity frontend to Python backend and implement TherapistFilter.cs	GUI successfully retrieves and displays filtered therapist data from Flask	1.5h	30/08/2025	**C**
16	Write Criterion C development documentation	Completed development write-up with screenshots, technique explanations, and bibliography	3h	15/09/2025	**C**
17	Test all 7 success criteria using test plan from Criterion B	All filtering functionality verified including multi-filter combinations and error handling	1h	20/09/2025	**B/C**
18	Test error handling and performance of search results	Error messages display correctly; search results consistently generated within 5 seconds	1h	20/09/2025	**C**
19	Record demonstration video of product functioning	5-7 minute video demonstrating all success criteria including edge cases and error handling	1.5h	28/09/2025	**D**
20	Hand over product to client and provide usage instructions	Client able to run TherapySort independently with Python backend and Unity frontend	0.5h	15/02/2026	**D**
21	Client evaluates product against success criteria via Google Forms questionnaire	Completed client feedback questionnaire with responses to all success criteria	0.5h	01/03/2026	**E**
22	Write Criterion E evaluation and recommendations	Completed evaluation referencing client feedback and all success criteria; minimum 2 realistic recommendations	2h	16/03/2026	**E**

Criterion C – Development

Initially, therapist data was loaded directly from a CSV file into the Unity GUI. This approach worked initially, but became difficult to maintain and made the code harder to debug. Additionally, it did not scale well as I added more fields, as with a CSV the searching application would have needed to iterate over the entirety of the file in order to find specific terms. To solve this, I moved the data into a proper SQLite database (therapySort.db) and used a Python Flask API (backend.py). This change meant that Unity only needed to request filtered data over HTTP, and all searching and validation logic could be centralised on the backend.

File formatting scripts

`init_db.py`

```python
import sqlite3
import os

DB_PATH = "therapySort.db"


print("Current working directory:", os.getcwd())
print("DB will be created at:", os.path.abspath(DB_PATH))

conn = sqlite3.connect(DB_PATH)
print("Connected to SQLite")


cur = conn.cursor()

cur.execute("DROP TABLE IF EXISTS therapists;")
print("Dropped old table (if existed)")

cur.execute("""

CREATE TABLE therapists (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    location TEXT,
    qualifications TEXT,
    verification TEXT,
    endorsements INTEGER,
    availability TEXT,
    experience_years INTEGER,
    sessions_completed INTEGER,
    description TEXT,
    job_title TEXT,
    phone TEXT,
    accepting_clients TEXT
);
""")
print("Created therapists table")


conn.commit()
conn.close()

print("Database initialized and therapists table created.")
```

Figure 9: SQLite database initialization script (init_db.py)

This script initialises the SQLite database and defines the therapist table schema. SQLite was chosen over other alternatives (MySQL, PostgreSQL) `sqlite3` being a part of Python's standard library made implementation straightforward and reduced unnecessary dependencies. Other benefits such as it being serverless and everything being stored in a single `.db` file made it suitable for a single-user desktop application.

The "`DROP TABLE IF EXISTS`" DDL statement made it easy for me to re-run the program when needed, for instance, I forgot a section in the .csv file used, but I was able to re-initialize it using the same scripts with no error.

Certain fields (`endorsements`, `experience_years`) used integer datatypes rather than text because the backend uses numeric comparison queries (eg. `endorsements >= ?`). All other fields are stored as `TEXT` since they are only ever searched through partial string matching. The name field is the only column with a `NOT NULL` constraint, as a therapist profile without a name would not be of any use to the user. Each record is also assigned a unique identifier (`ID`) using `AUTOINCREMENT`, allowing the backend to reference individual therapists unambiguously. Finally, conn.commit() persists the schema to disk and conn.close() releases the file lock, ensuring the database is ready before `importCsvSqlite.py` runs.

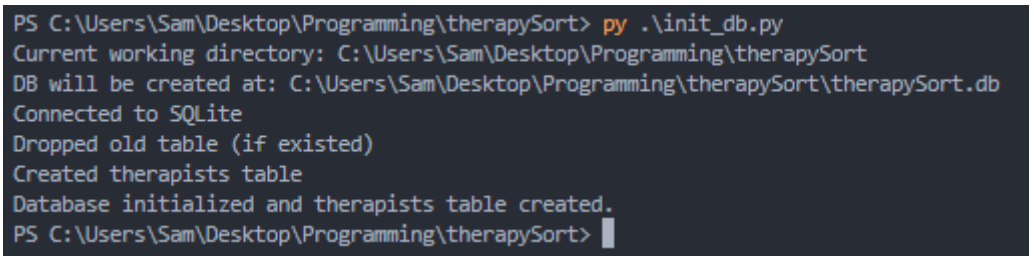


Figure 10: Console/Terminal output that confirms whether the database and table were created

`importCsvSqlite.py`

```python
import sqlite3
import pandas as pd

DB_PATH = "therapySort.db"
CSV_PATH = "mental_health_dataset.csv"

df = pd.read_csv(CSV_PATH)

df = df.rename(columns={
    "Name": "name",
    "Location": "location",
    "Qualifications": "qualifications",
    "Verification": "verification",
    "Endorsements": "endorsements",
    "Availability": "availability",
    "Experience_Years": "experience_years",
    "Sessions_Completed": "sessions_completed",
    "Description": "description",
    "Job_Title": "job_title",
    "Phone": "phone",
    "Accepting_Clients": "accepting_clients"
})

conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()

for _, row in df.iterrows():
    cursor.execute("""
        INSERT INTO therapists (name, location, qualifications, verification,
        endorsements, availability, experience_years, sessions_completed,
        description, job_title, phone, accepting_clients)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, (
        row.get("name", ""),
        row.get("location", ""),
        row.get("qualifications", ""),
        row.get("verification", ""),
        int(row.get("endorsements", 0)) if pd.notna(row.get("endorsements")) else None,
        row.get("availability", ""),
        int(row.get("experience_years", 0)) if pd.notna(row.get("experience_years")) else None,
        row.get("sessions_completed", ""),
        row.get("description", ""),
        row.get("job_title", ""),
        row.get("phone", ""),
        row.get("accepting_clients", "")
    ))

conn.commit()
conn.close()
print("Data imported into therapists table from CSV.")
```

Figure 11: Importing of CSV data into SQL database.

This script reads the mock CSV dataset and inserts each record into the database created by `init_db.py`. Pandas was chosen over the built-in CSV module, as it handles missing values and misplaced rows better.

`df.rename()` is used to normalise the CSV headers into lowercase. There are other formatting checks like this, such as `row.get("field", "")`, which sets the string to empty by default which prevents KeyErrors.

`pd.notna()` is used specifically before the `int()` cast because pandas represents missing numeric values as floats (or Not a Number/NaN), which would return a ValueError. This check is therefore casting `endorsements` and `experience_years`.

To prevent SQL injection, values are passed separately from the SQL string instead of concatenating them into it directly.

```
PS C:\Users\Sam\Desktop\Programming\therapySort> py .\importCsvSqlite.py
Data imported into therapists table from CSV.
PS C:\Users\Sam\Desktop\Programming\therapySort>
```

Figure 12: Terminal output confirming data importation.

Rows: 1,000

	id	name	location	qualifications	verificati...	endor
1	1	Alice Smith	Canada	MA Counseling	Unverified	
2	2	Alice Smith	UK	MA Counseling	Unverified	
3	3	Carol White	USA	MSc Psychotherapy	Unverified	
4	4	Bob Jones	Online Only	PhD Clinical Psychology	Verified	
5	5	Bob Jones	Canada	MSc Psychotherapy	Unverified	
6	6	Bob Jones	UK	MSc Psychotherapy	Unverified	
7	7	Alice Smith	UK	MA Counseling	Unverified	
8	8	Emma Wilson	UK	MA Counseling	Verified	

Figure 13: Snippet of therapists database, populated with 1000 rows.

Main scripts

backend.py

This is a Python Flask server that exposes a single /filter endpoint. When a GET request is received, it reads the query parameters provided by the frontend and dynamically constructs a SQL WHERE clause based on the user's input (i.e. which filters are active). This executes the query against the SQLite database. If the user doesn't include a parameter, it is simply left blank and not appended to the list of conditions used to stack multiple filters. I chose this method because writing a separate SQL query for every possible filter combination would be impractical, especially with eight filterable fields. Instead, each parameter is checked individually, and if present, its corresponding condition is appended to the conditions list. The results are returned as a JSON array of the therapist records.

```python
if min_endorsements:
    try:
        min_val = int(min_endorsements)
        conditions.append("endorsements >= ?")
        values.append(min_val)
    except ValueError:
        return jsonify({"error": "endorsements must be an integer"}), 400

if keyword:
    # match keyword in multiple text columns
    conditions.append("""(
        LOWER(description) LIKE ?
        OR LOWER(job_title) LIKE ?
        OR LOWER(qualifications) LIKE ?
    )""")
    kw = "%" + keyword.lower() + "%"
    values.extend([kw, kw, kw])
```

Figure 14: Snippet of parameter appending. Appropriate error messages are displayed for improper datatypes for the integers, and the keyword searches through the description as well as the job title, and qualifications for matches.

```python
if job_title:
    conditions.append("LOWER(job_title) LIKE ?")
    values.append("%" + job_title.lower() + "%")

if accepting_clients:
    conditions.append("LOWER(accepting_clients) = ?")
    values.append(accepting_clients.lower())

# (optional) WHERE combination
if conditions:
    base_query += " WHERE " + " AND ".join(conditions)

# (optional) adding limit
if limit:
    try:
        limit_val = int(limit)
        base_query += " LIMIT ?"
        values.append(limit_val)
    except ValueError:
        return jsonify({"error": "limit must be an integer"}), 400
```

Figure 15: Other filter parameters that can be appended, as well as the WHERE combination to filter for multiple parameters.

```
PS C:\Users\Sam\Desktop\Programming\therapySort> py backend.py
 * Serving Flask app 'backend'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 131-793-466
```

Figure 16: Terminal output when backend server runs. It is hosted locally on a flask server.

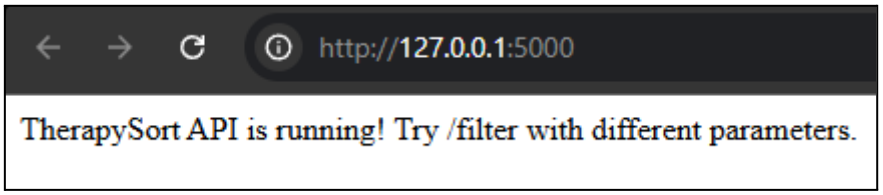


Figure 17: In-browser view of the flask server (no GUI).

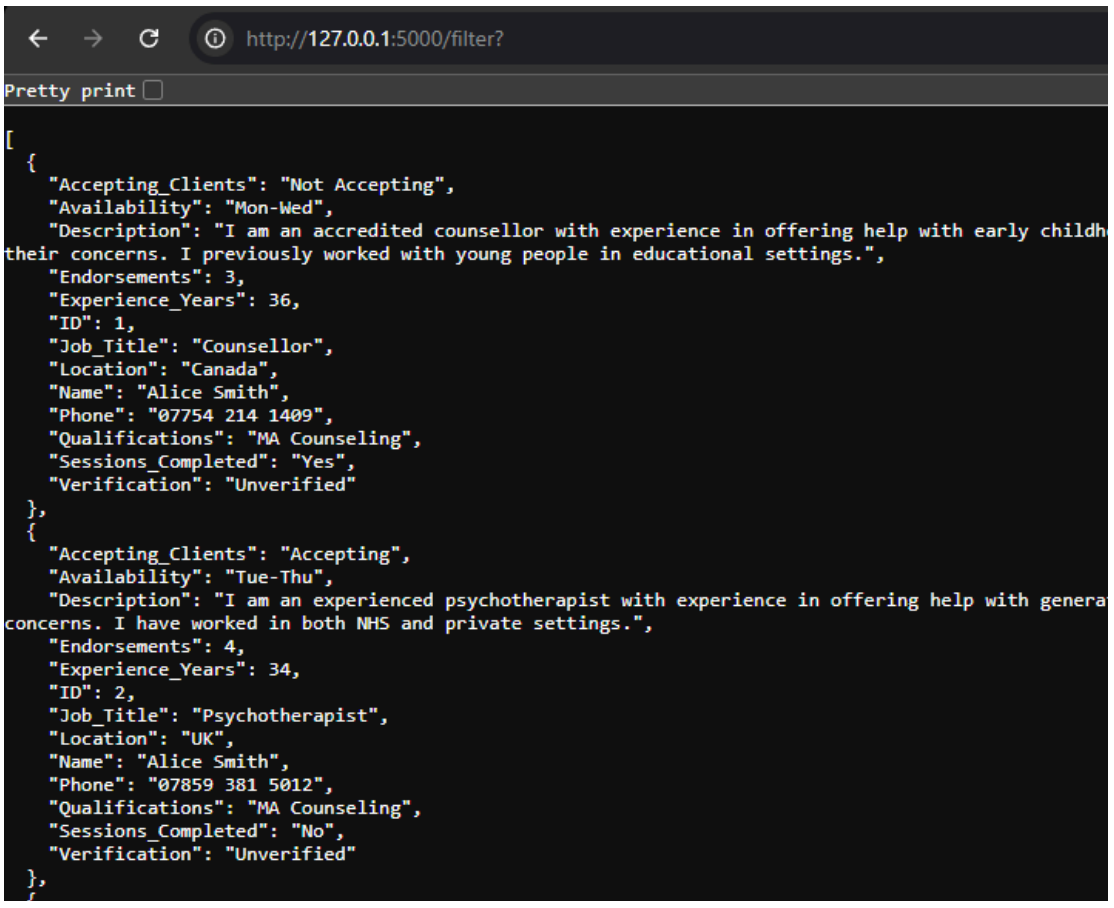


Figure 18: In-browser view of flask server with filter query.

HTTP_Test.cs

A small Unity script was made before creating the GUI to confirm that Unity could successfully send GET requests to the Flask server and receive JSON responses.

```
IEnumerator TestConnection()
{
    string url = "http://127.0.0.1:5000/"; // local host

    UnityWebRequest request = UnityWebRequest.Get(url);
    yield return request.SendWebRequest();

    if (request.result == UnityWebRequest.Result.Success)
    {
        Debug.Log("Connected! Response: " + request.downloadHandler.text);
    }
    else
    {
        Debug.Log("Error: " + request.error);
    }
}
```

Figure 19: Test script to verify Unity-Flask connection.

TherapistFilter.cs

The main Unity script that gathers filter inputs from the user, constructs a request URL, and sends it to the backend. The JSON returned by Flask is parsed into C# objects and displayed as UI cards. The script uses a card generation system that instantiates prefabs and fills text fields with therapist information, including fallback values for missing data. I also fixed several issues during development, which are discussed in the later “Errors Encountered” section.

```
private string BuildFilterUrl()
{
    string url = backendUrl + "?";
    Debug.Log("Requesting URL: " + url);

    bool firstParam = true;

    void AddParam(string key, string value)
    {
        if (!string.IsNullOrEmpty(value))
        {
            if (!firstParam)
            {
                url += "&";
            }

            url += key + "=" + UnityWebRequest.EscapeURL(value);
            firstParam = false;
        }

    }

    void AddToggleParam(string key, Toggle toggle, string valueIfOn)
    {
        if (toggle != null && toggle.isOn)
        {
            AddParam(key, valueIfOn);
        }
    }

    AddParam("location", locationInput != null ? locationInput.text : "");
    AddParam("availability", availabilityInput != null ? availabilityInput.text : "");
    AddParam("qualifications", qualificationsInput != null ? qualificationsInput.text : "");
    AddToggleParam("verification",  verificationToggle, "Verified");
    AddParam("endorsements", endorsementsMinInput != null ? endorsementsMinInput.text : ""); // min rating
    AddParam("limit", limitInput != null ? limitInput.text : "");
    AddParam("keyword", keywordInput != null ? keywordInput.text : "");
    AddParam("job_title", jobTitleInput != null ? jobTitleInput.text : "");
    AddParam("accepting_clients", acceptingClientsInput != null ? acceptingClientsInput.text : "");

    return url;

}
```

Figure 20: Method to build filter parameter request through the backend url.

The BuildFilterUrl() method uses a small helper function AddParam defined inside it, which handles whether to add a ? or a & depending on whether it's the first parameter. This avoided me having to clean up the URL string afterwards. I also used UnityWebRequest.EscapeURL on each value before adding it to the URL, which sanitises any special characters the user might type in. This works alongside the parameter queries in the backend as a second layer of protection against malformed input.

```
// retrieves filtered therapists from given url
1 reference
IEnumerator GetFilteredTherapists()
{
    resultsText.text = "Searching ...";

    string url = BuildFilterUrl();
    // Debug.Log("Requesting " + url);

    using (UnityWebRequest request = UnityWebRequest.Get(url))
    {
        yield return request.SendWebRequest();

        bool error = request.result != UnityWebRequest.Result.Success;

        if (error)
        {
            resultsText.text = "Error: " + request.error;
            yield break;
        }

        string jsonResponse = request.downloadHandler.text;

        DisplayResults(jsonResponse);

    }

}
```

Figure 21: Method to retrieve filtered therapists from local host and display appropriate processing and error messages.

One thing worth explaining is the use of an IEnumerator for GetFilteredTherapists rather than a regular method. Unity runs everything on one thread, so if I made a normal HTTP request it would freeze the whole application until the server responded. Using a coroutine with yield return request.SendWebRequest() pauses the method and lets Unity keep running normally, then resumes once the response comes back. This keeps the interface responsive while waiting for Flask to reply.

```
private void DisplayResults(string jsonResponse)
{

    // wrap raw array (from Flask) into an object so JsonUtility can parse it
    string wrappedJson = "{\"therapists\":" + jsonResponse + "}";

    TherapistList therapistList;

    try
    {
        therapistList = JsonUtility.FromJson<TherapistList>(wrappedJson);
        Debug.Log("Parsed therapists: " + (therapistList?.therapists == null ? "null" : therapistList.therapists.Length.ToString()));
    }
    catch (System.Exception e)
    {
        Debug.LogError("JSON parse error: " + e.Message);
        resultsText.text = "Sorry, couldn't read results.";
        return;
    }

    // Clear previous cards
    foreach (Transform child in resultsContainer)
    {
        Destroy(child.gameObject);
    }
```

Figure 22: JSON wrapping and parsing method.

```
// Make cards
foreach (Therapist t in therapistList.therapists)
{
    GameObject card = Instantiate(therapistCardPrefab, resultsContainer);

    // find text elements on the prefab
    TMP_Text nameField = card.transform.Find("NameText")?.GetComponent<TMP_Text>();
    TMP_Text qualField = card.transform.Find("QualificationsText")?.GetComponent<TMP_Text>();
    TMP_Text locField = card.transform.Find("LocationText")?.GetComponent<TMP_Text>();
    TMP_Text availField = card.transform.Find("AvailabilityText")?.GetComponent<TMP_Text>();
    TMP_Text metaField = card.transform.Find("MetaText")?.GetComponent<TMP_Text>();
    TMP_Text jobTitleField = card.transform.Find("JobTitleText")?.GetComponent<TMP_Text>();
    TMP_Text phoneField = card.transform.Find("PhoneText")?.GetComponent<TMP_Text>();
    TMP_Text clientsField = card.transform.Find("AcceptingClientsText")?.GetComponent<TMP_Text>();
    TMP_Text descField = card.transform.Find("DescriptionText")?.GetComponent<TMP_Text>();
    TMP_Text experienceField = card.transform.Find("ExperienceText")?.GetComponent<TMP_Text>();

    // fill them with clean labels (for aesthetics)
    if (nameField != null)
        nameField.text = SafeText(t.Name, "Name not listed");

    if (qualField != null)
        qualField.text = SafeText(t.Qualifications, "Qualifications not listed");
```

Figure 23: Snippet of prefab population loop using text elements.

```
private string SafeText(string s, string fallback)
{
    if (string.IsNullOrEmpty(s)) return fallback;
    return s;
}
```

Figure 24: String to prevent therapist cards from showing empty labels when data is missing from the database.

FilterDropdownController.cs

To make filtering clearer and more manageable, I redesigned the interface from plain text boxes to grouped toggles and dropdowns (e.g., Verified Only, Minimum Endorsements). These controls write to the same variables that the filtering script uses, allowing multiple filters to be active at once.

```
private void BindToggle(Toggle toggle, GameObject group)
{
    if (toggle == null || group == null) return;

    // make sure the group matches the toggle's initial state
    toggle.isOn = false;
    group.SetActive(false);

    GameObject capturedGroup = group;

    toggle.onValueChanged.AddListener((isOn) =>
    {
        if (isOn)
        SetAllGroupsActive(false); // all other groups hidden first

        capturedGroup.SetActive(isOn);

        // clear inputs when filter deactivated
        if (!isOn)
            ClearGroupInputs(capturedGroup);
    });

}
```

Figure 25: Method to harmonise toggles and groups. Clears a given input when the associated filter (toggle) is deactivated.

The BindToggle method uses onValueChanged.AddListener to register a callback that only fires when a toggle is actually switched, rather than checking every toggle's state on every frame, as this would waste resources. Moreover, this is more efficient and keeps the filter UI logic separate from the search logic. The lambda uses capturedGroup rather than group directly because by the time the callback fires, the loop that called BindToggle has already moved on; capturing it in a new variable makes sure the right group is referenced.

```
private void ClearGroupInputs(GameObject group)
{

    foreach (var inputField in group.GetComponentsInChildren<TMP_InputField>(true))
        inputField.text = "";

}
```

Figure 26: Method to clear input fields for all given groups.

```
private void SetAllGroupsActive(bool active)
{
    SetGroupActive(locationGroup, active);
    SetGroupActive(availabilityGroup, active);
    SetGroupActive(qualificationsGroup, active);
    SetGroupActive(verificationGroup, active);
    SetGroupActive(endorsementsGroup, active);
    SetGroupActive(keywordGroup, active);
    SetGroupActive(limitGroup, active);
    SetGroupActive(jobTitleGroup, active);
    SetGroupActive(phoneGroup, active);
    SetGroupActive(acceptingClientsGroup, active);
}
```

Figure 27: Method to set all groups active.

GUI Creation

Figure 28: Preliminary GUI design used to decide filter option placement and therapist display scale.

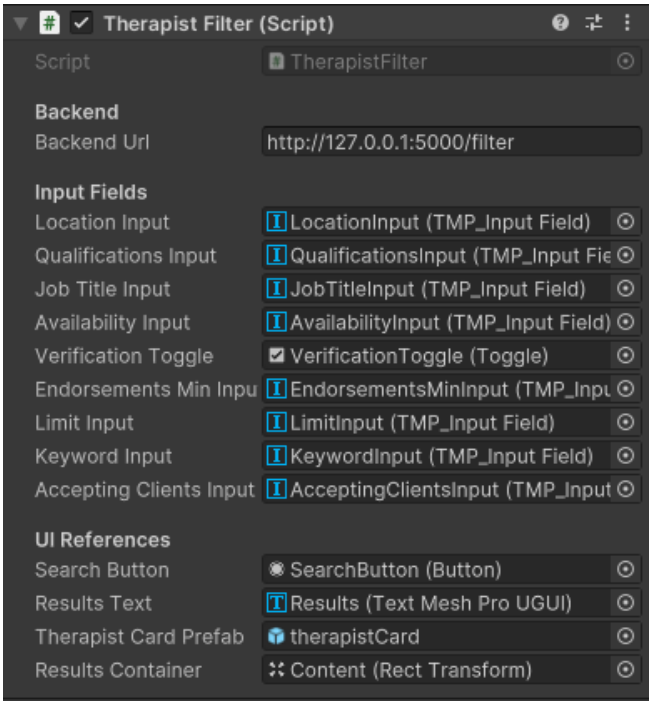


Figure 29: Inspector for the “Therapist Filter” script. All input fields and UI references (such as buttons and the therapist card prefab) can be seen.

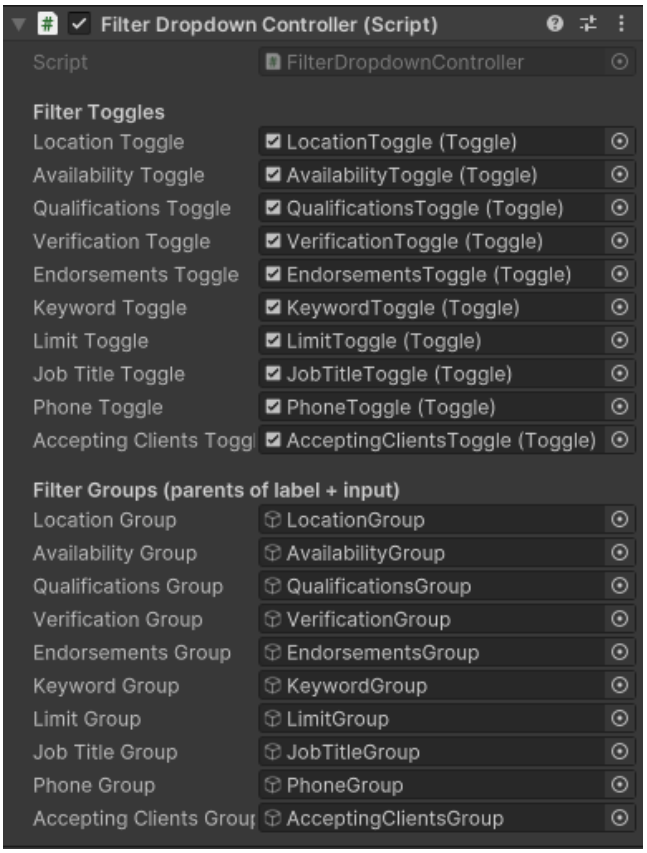


Figure 30: Inspector for the “Filter Dropdown Controller” script. All toggles and filter groups can be seen.

Figure 31: Unity Hierarchy for the TherapySort GUI.

Figure 32: Final design for the user interface (GUI).

Errors Encountered

- When the backend script was not run the Unity-end would return an error.

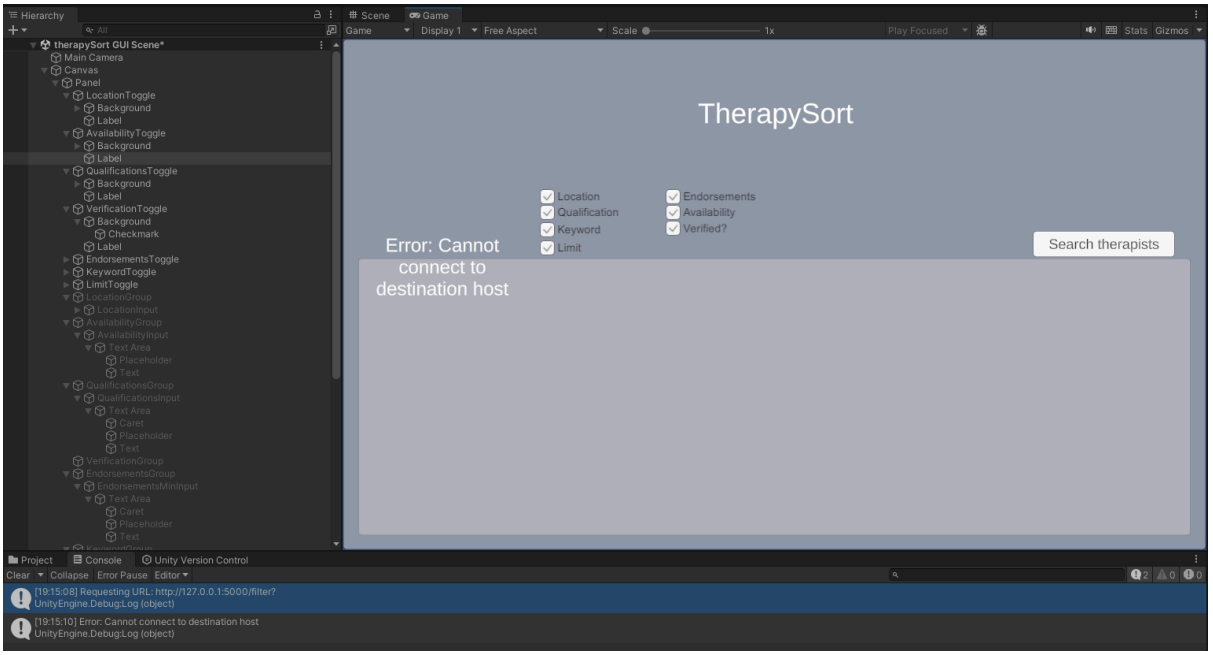


Figure 33: Screenshot of host connection error.

- After compilation, the “No therapists matched your filters” message would be displayed despite no filters being specified. This error occurred because the default toggle state in Unity is on. The fix for this was to untick the “isOn” setting in the inspector for each toggle.

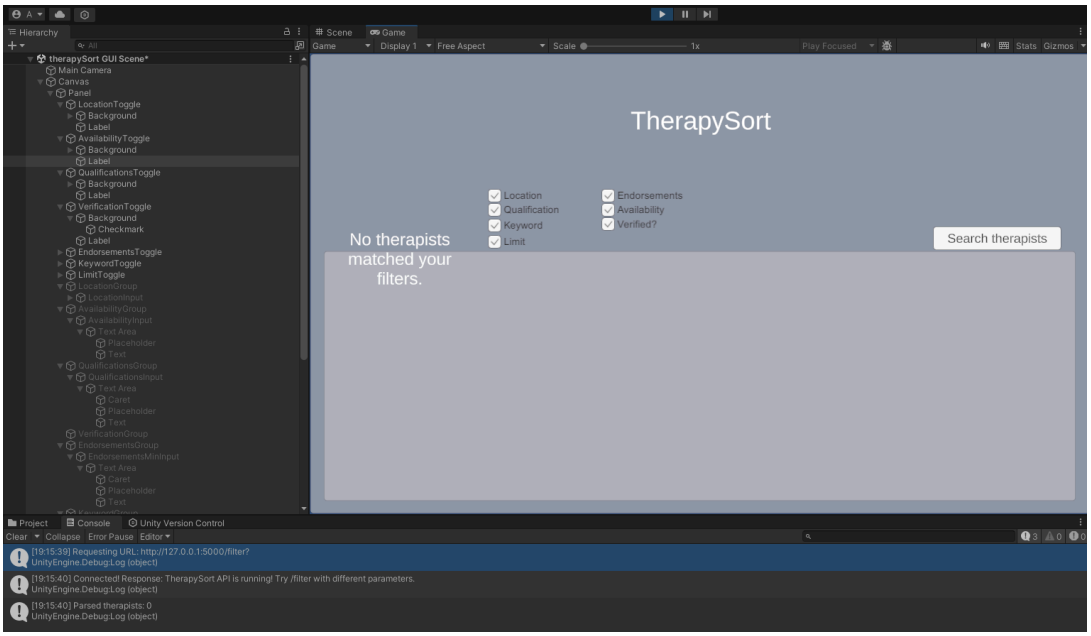


Figure 34: Screenshot of default toggle state error.

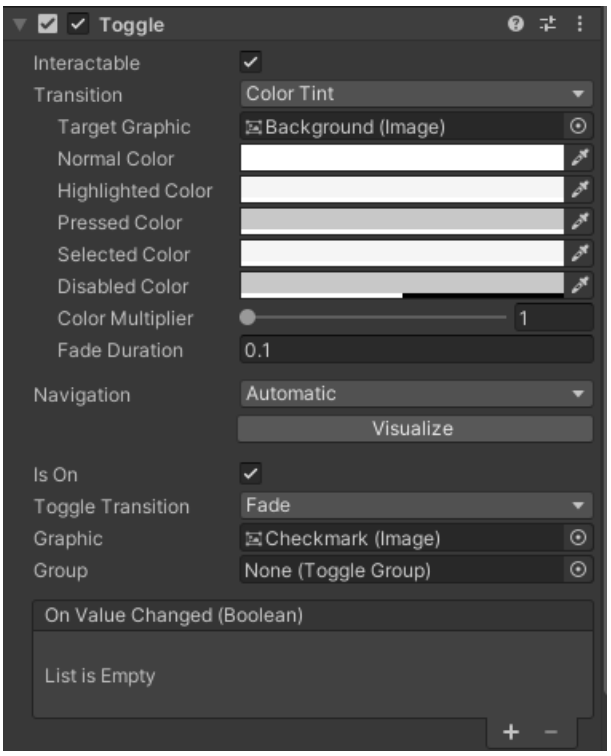


Figure 35: Screenshot toggle inspector component. The “isOn” option can be seen ticked by default.

- I was going to implement blank image cards as a stylistic choice for each therapist, however this proved to come with some formatting issues. I tried adding a vertical layout group and content size fitter, but ultimately the debugging time took too long so the idea was abandoned.

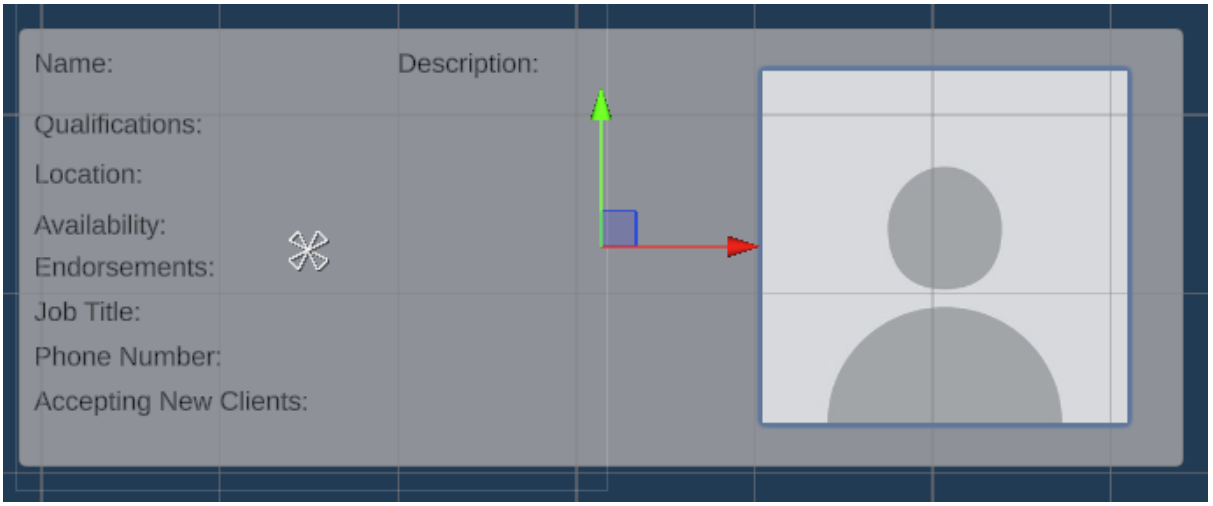


Figure 36: Initial therapist prefab card design (with blank therapist image)

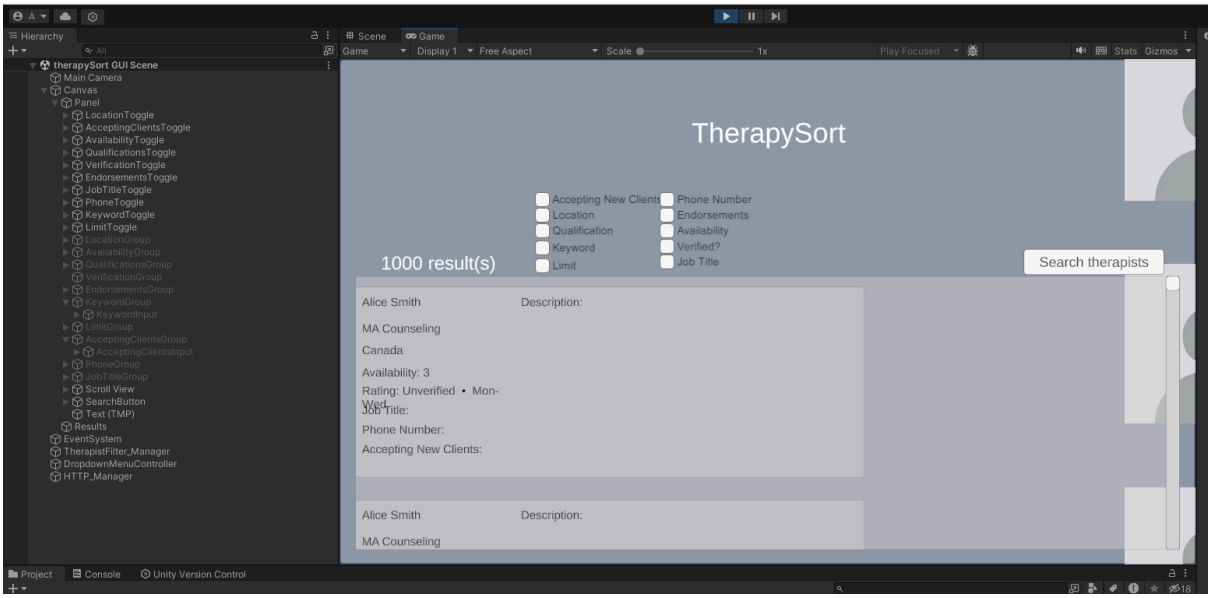


Figure 37: Screenshot of display issue when including blank therapist images.

- Misspellings occasionally occurred which caused certain input fields or toggles to be misread upon initial configuration.

Figure 38: Error caused by misspelling “experienceField” as “expereienceField”.

[Words: 1087]

Criterion E – Evaluation

Success Criterion	Met?	Client Feedback
Data imports correctly	Yes	Confirmed via questionnaire (Appendix 4)
Filter by all parameters	Yes	All filters worked as expected
Multiple simultaneous filters	Yes	Confirmed working correctly
Clear GUI with filter controls	No	“UI could be arranged better” - Mr. Rajabov (Appendix 4)
Specify number of results	Yes	Limit input worked correctly
Error messages display	Yes	Clear messages shown for invalid input
Results within 5 seconds	Yes	Well within threshold in all tested cases

Considerations and Improvements

Mr. Rajabov’s critiques (Appendix 4) will be discussed alongside potential improvements in this section.

UI arrangement
Mr. Rajabov stated that the UI arrangement was unideal. He specifically cites the placement of the filters, and the title space being too large. In hindsight, the large amount of title space could have been used to better arrange the displayed therapists, as Mr. Rajabov says. Displaying two therapists per row could have been a good use of this space, as it would be easier for the user to display a large number of therapists without the display becoming overwhelming for them.

Keyword filter functionality
Mr. Rajabov found the keyword filter “a little weird”, stating that the filter could have been applied by the user accessing the search input field in the first place. This is an incredibly valid critique, as most modern search engines treat text entered into a search bar as an implicit keyword search, without requiring the user to toggle a separate filter. A more intuitive implementation would be automatically applying a keyword search whenever the search box is filled with user input, removing the need for an extra filter/toggle entirely. This would remove the need for the number of steps required for the user to access this filter.

Therapist profiles
Mr. Rajabov noted that the therapist “Alice Smith” appeared twice in the demo results, as she had created separate listings for counselling and psychotherapy services. This is a limitation of the mock dataset rather than filtering logic itself, however it highlights a usability issue that could arise if live data were to be used. Duplicate profiles could be detected and merged by grouping entries that share the same name and phone number, for instance. Additionally, Mr. Rajabov suggested condensing the information displayed on each card and implementing an expandable panel (similar to the layout of Microsoft Outlook; an example can be viewed in Appendix 7). Where clicking a therapist reveals their full description in a side panel. This would allow for significantly more profiles to be visible simultaneously, and make the interface more digestible for the user.

Limiting number of therapists
Mr. Rajabov suggested that a range slider would have been a more intuitive implementation for limiting the number of displayed therapists than the current text input field (similar to the Amazon price filter; an example can be viewed in Appendix 8). A dual-handle slider allowing the user to set both a minimum and maximum number of results would give greater control and be more immediately understandable to a non-technical user.

These improvements were outside the original scope of the project, but provide a clear direction for future development.

[Words: 440]

Bibliography

Datablist. 2025. *Download Sample CSV Files*. Visited 08/08/2025. https://www.datablist.com/learn/csv/download-sample-csv-files#download-people-sample-csv-files.*

Pallets Projects. 2025. *Flask*. Visited 10/07/2025. https://flask.palletsprojects.com

pandas development team. 2025. *pandas documentation*. Visited 09/08/2025. https://pandas.pydata.org

Python Software Foundation. 2025. *sqlite3 — DB-API 2.0 interface for SQLite databases*. Visited 09/08/2025. https://docs.python.org/3/library/sqlite3.html

Unity Technologies. 2025. *Unity Engine*. Visited 10/07/2025. https://unity.com

Unity Technologies. 2025. *TextMeshPro*. Visited 20/08/2025. https://docs.unity3d.com/Packages/com.unity.textmeshpro@3.0/manual/index.html

**The therapist dataset was sourced from Datablist and modified to include all filter parameters specified by the client, including verification status, endorsements, availability, and job title*.

Appendices

Appendix 1: Transcript of initial meeting with Mr. Rajabov

Client: Baseer Rajabov

Date: 23 January 2025
Meeting Type: Initial requirements discussion

Transcript:

Developer: Hi Baseer, I'm looking for a project to build for my Computer Science coursework. Is there anything that could make your life easier that could potentially be solved through code?

Client: Hi. Actually yes, there is something that could be useful. I often look through those websites with therapists, like BetterHelp, and it takes a lot of time to manually search through profiles.

Developer: That makes sense. What kind of program do you think could help with that?

Client: I found a website that lists therapists with their details – I can't remember the name but if I find it I'll send it to you in an email. Ideally, I'd like a program that scrapes the information from the therapists and organizes it so I can search through it more easily.

Developer: Is there specific information you want to be scraped?

Client: Maybe things like the therapist's name, phone number, qualifications, professional role (for example counsellor or psychotherapist), location, etc…. It would be helpful if the program stored everything and then allowed me to search through it.

Developer: What kinds of searches would you want to perform?

Client: I'd like to filter by job type, location, qualifications, that sort of thing. Basically whatever information you can get from the website.

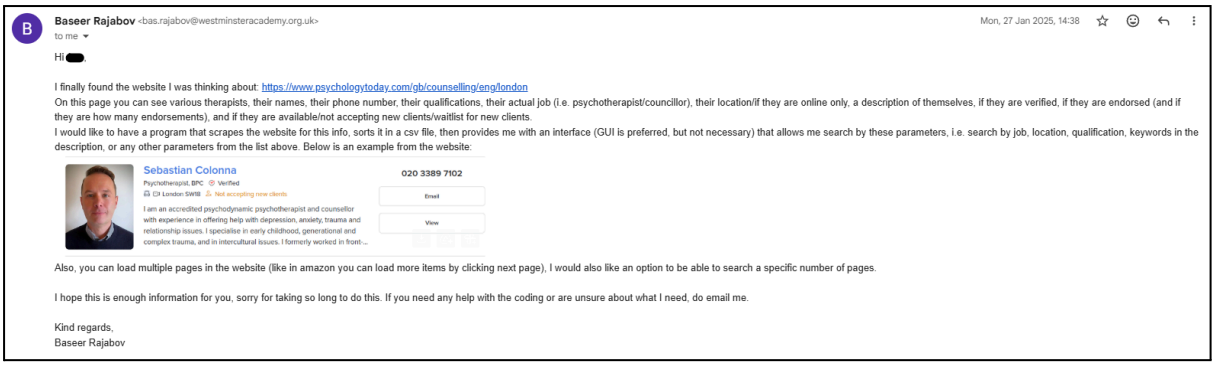
Developer: Would it help if you could choose how many pages the program collects?

Client: Yes, that would be useful so the program doesn't have to scrape the entire site every time.

Developer: Okay, so to summarise, I'll design a system that scrapes the therapist listings, stores them in a CSV file, and provides a way to search and filter the results.

Client: That sounds great. I think that would make the process much faster.

Appendix 2: Email correspondence with Mr. Rajabov



Baseer Rajabov <bas.rajabov@westminsteracademy.org.uk> Mon, 27 Jan 2025, 14:38

to me

Hi [redacted]

I finally found the website I was thinking about: https://www.psychologytoday.com/gb/counselling/eng/london

On this page you can see various therapists, their names, their phone number, their qualifications, their actual job (i.e. psychotherapist/councillor), their location/if they are online only, a description of themselves, if they are verified, if they are endorsed (and if they are how many endorsements), and if they are available/not accepting new clients/waitlist for new clients.

I would like to have a program that scrapes the website for this info, sorts it in a csv file, then provides me with an interface (GUI is preferred, but not necessary) that allows me search by these parameters, i.e. search by job, location, qualification, keywords in the description, or any other parameters from the list above. Below is an example from the website:

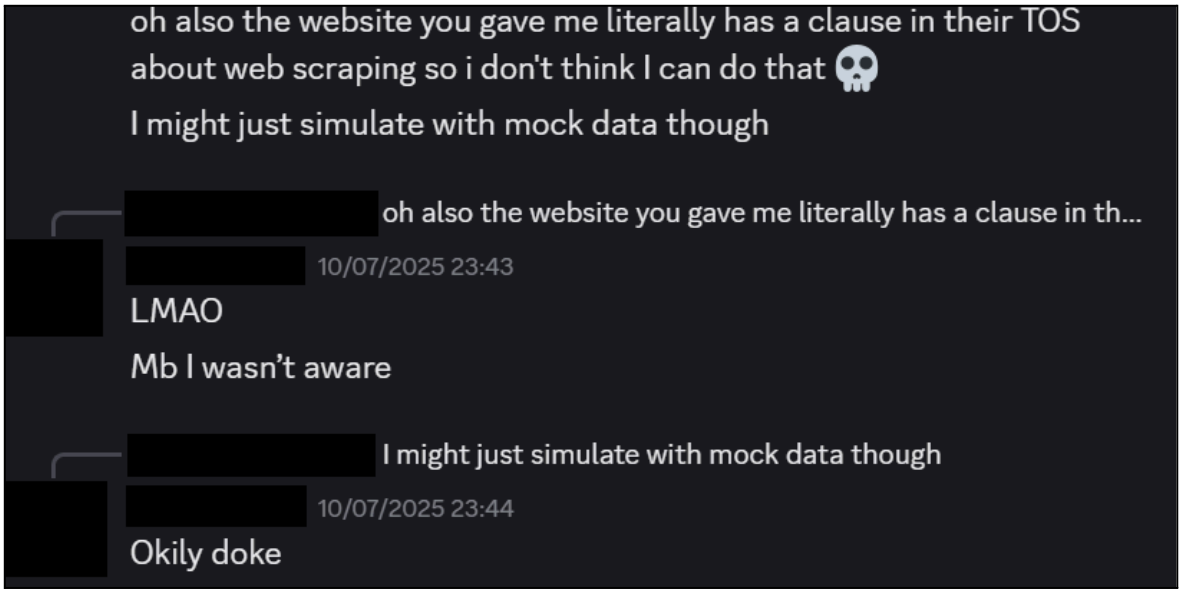
Also, you can load multiple pages in the website (like in amazon you can load more items by clicking next page), I would also like an option to be able to search a specific number of pages.

I hope this is enough information for you, sorry for taking so long to do this. If you need any help with the coding or are unsure about what I need, do email me.

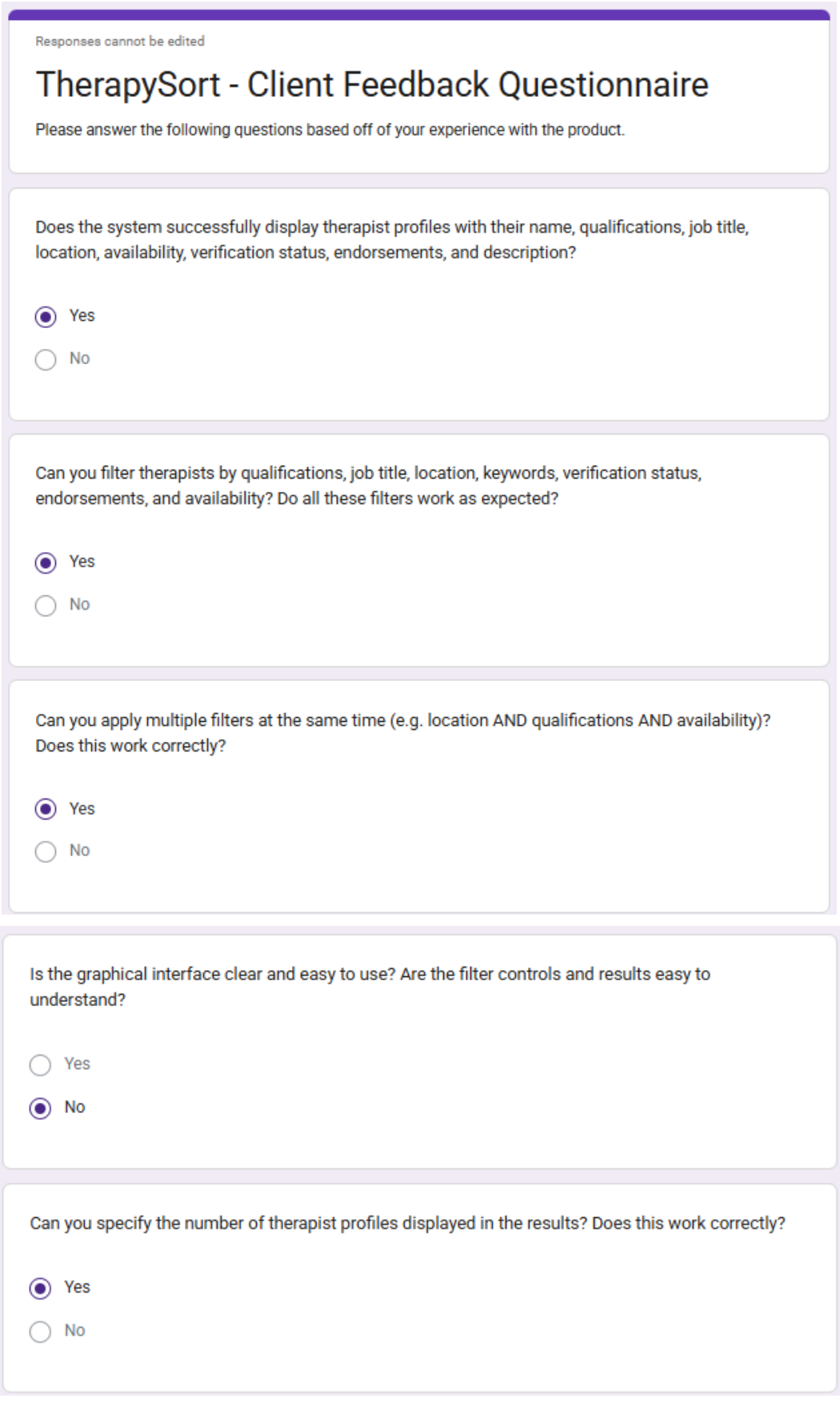
Kind regards,
Baseer Rajabov

Appendix 3: Communication regarding website scraping and use of a mock dataset

Screenshot of client communication confirming that scraping therapist data from the suggested website would violate its terms of service and that the system should instead use a mock dataset with the same parameters.



Appendix 4: Google Forms Questionnaire filled out by Mr. Rajabov.



Responses cannot be edited

TherapySort - Client Feedback Questionnaire

Please answer the following questions based off of your experience with the product.

Does the system successfully display therapist profiles with their name, qualifications, job title, location, availability, verification status, endorsements, and description?

- (●) Yes
- ( ) No

Can you filter therapists by qualifications, job title, location, keywords, verification status, endorsements, and availability? Do all these filters work as expected?

- (●) Yes
- ( ) No

Can you apply multiple filters at the same time (e.g. location AND qualifications AND availability)? Does this work correctly?

- (●) Yes
- ( ) No

Is the graphical interface clear and easy to use? Are the filter controls and results easy to understand?

- ( ) Yes
- (●) No

Can you specify the number of therapist profiles displayed in the results? Does this work correctly?

- (●) Yes
- ( ) No

When you enter invalid input or no therapists match your filters, does the system display a clear error or message?

- (●) Yes
- ( ) No

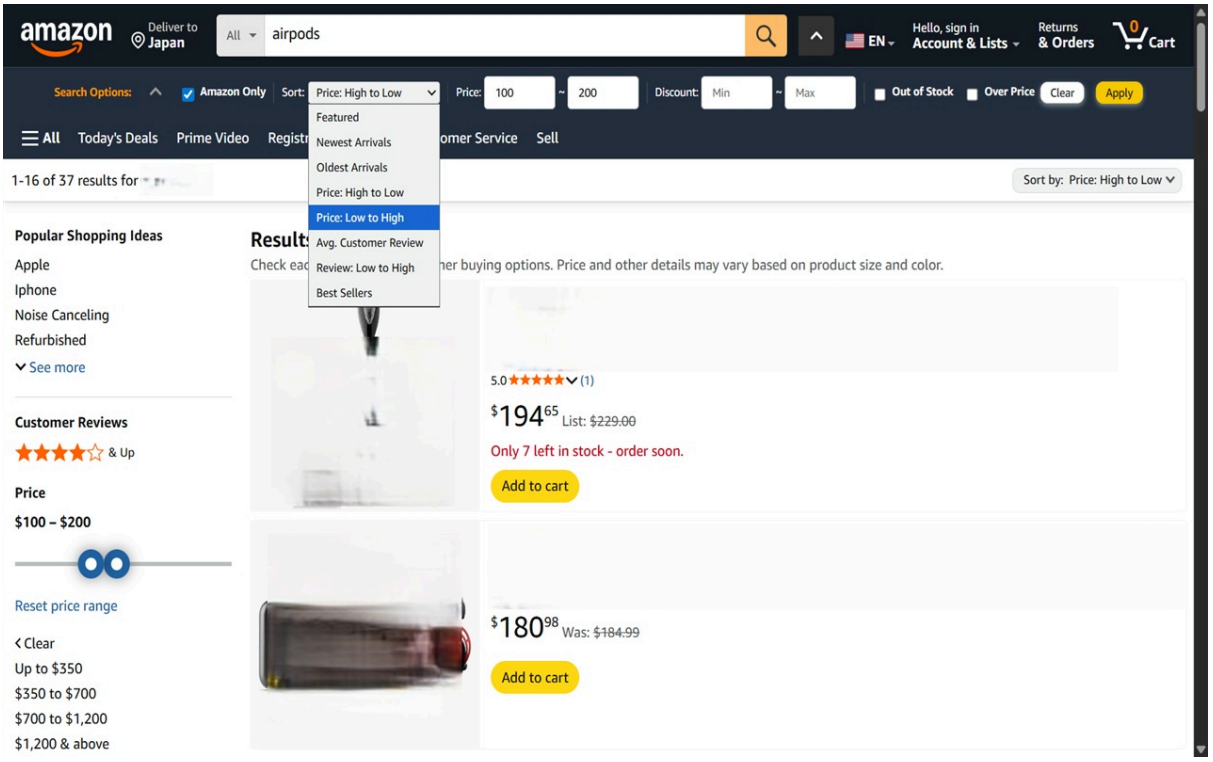
Do search results appear quickly (within approximately 5 seconds)?

- (●) Yes
- ( ) No

Is there anything you feel is missing or could be improved?

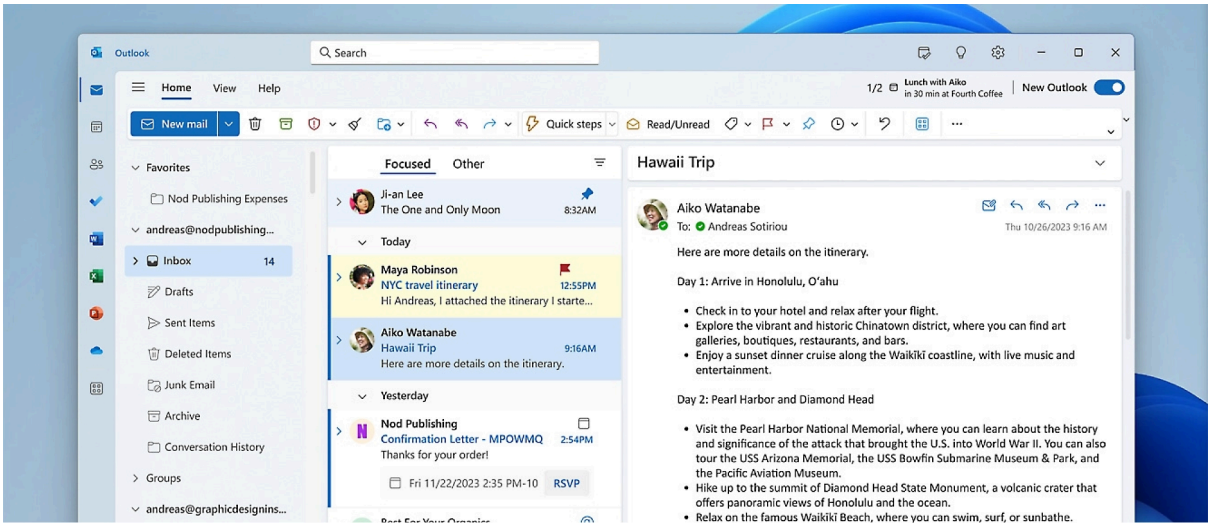
The ui could be arranged better, for example the number of results doesn't need to be that big or in that place. The placing of the filters is also not ideal, it might have been better to make the TherapySort title space smaller and use the increased vertical space to list therapists next to each other, so 2 therapists in every row. The keyword filter I thought was a little weird as opposed to just applying the keyword filter if I use the search function at all, needing to type in a keyword and toggle a keyword filter and hit enter is a bit tedious. Additionally, the first result in the demo video 'Alice Smith' comes up twice as she made a listing for counselling services and psychotherapy services. Assuming this happens multiple times, you could condense the information shown by just showing the info of the person without their descriptive paragraph, and have the paragraph show if you click on them. This way you could fit more information in and have it more digestible for the user. You could also have it as a column of this condensed information, with a side panel for the paragraphs kinda like MS outlook where the emails are on the right, and when one is opened the right half of the screen displays their descriptive paragraph. Also I'm not sure if these are from mulitple websites or only one, but you could include a button somewhere to the listing from the website. For limiting the number of therapists, I feel like a slider where you can change the max and minimum (like the price slider when filtering on Amazon) would have been a better implementation than the search box.

Appendix 5: Amazon UI example. The price slider can be seen on the left side of the screenshot.



Source:https://lh3.googleusercontent.com/eC2EHN_ahpko0Vp11GNj-yrtRrvIj6IQQEdaWJgykS4FCtQ0Fj6Um61MzIj5tAfxa2CM3wdSXRBWKu8GLwa-KgZP=s1280-w1280-h800

Appendix 6: Outlook UI example.



Source:https://cdn-dynmedia-1.microsoft.com/is/image/microsoftcorp/benefits-img1-370473?resMode=sharp2&op_usm=1.5,0.65,15,0&wid=2000&hei=858&qlt=100&fit=constrain

Appendix 7: Full init_db.py script

```
import sqlite3
import os

DB_PATH = "therapySort.db"


print("Current working directory:", os.getcwd())
print("DB will be created at:", os.path.abspath(DB_PATH))

conn = sqlite3.connect(DB_PATH)
print("Connected to SQLite")


cur = conn.cursor()

cur.execute("DROP TABLE IF EXISTS therapists;")
print("Dropped old table (if existed)")

cur.execute("""

CREATE TABLE therapists (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    location TEXT,
    qualifications TEXT,
    verification TEXT,
    endorsements INTEGER,
    availability TEXT,
    experience_years INTEGER,
    sessions_completed INTEGER,
    description TEXT,
    job_title TEXT,
    phone TEXT,
    accepting_clients TEXT
);
""")
print("Created therapists table")


conn.commit()
conn.close()

print("Database initialized and therapists table created.")
```

Appendix 8: Full importCsvSqlite.py script

```
import sqlite3
import pandas as pd

DB_PATH = "therapySort.db"
CSV_PATH = "mental_health_dataset.csv"

df = pd.read_csv(CSV_PATH)

df = df.rename(columns={
    "Name": "name",
    "Location": "location",
    "Qualifications": "qualifications",
    "Verification": "verification",
    "Endorsements": "endorsements",
    "Availability": "availability",
    "Experience_Years": "experience_years",
    "Sessions_Completed": "sessions_completed",
    "Description": "description",
    "Job_Title": "job_title",
    "Phone": "phone",
    "Accepting_Clients": "accepting_clients"
})

conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()

for _, row in df.iterrows():
    cursor.execute("""
        INSERT INTO therapists (name, location, qualifications, verification,
        endorsements, availability, experience_years, sessions_completed,
        description, job_title, phone, accepting_clients)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, (
        row.get("name", ""),
        row.get("location", ""),
        row.get("qualifications", ""),
        row.get("verification", ""),
        int(row.get("endorsements", 0)) if pd.notna(row.get("endorsements")) else None,
        row.get("availability", ""),
        int(row.get("experience_years", 0)) if pd.notna(row.get("experience_years")) else
None,
        row.get("sessions_completed", ""),
        row.get("description", ""),
        row.get("job_title", ""),
        row.get("phone", ""),
        row.get("accepting_clients", "")
    ))

conn.commit()
conn.close()
print("Data imported into therapists table from CSV.")
```

Appendix 9: Full backend.py script

```
import random
random.seed(42) # loads random set of data each time

import pandas as pd
from flask import Flask, request, jsonify
from flask_cors import CORS
import sqlite3

DB_PATH = "therapySort.db"

app = Flask(__name__)
CORS(app)

def get_db_connection():
    # row_factory = dictionary (essentially) for easier access to columns by name
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

@app.route("/", methods=["GET"])
def index():
    return "TherapySort API is running! Try /filter with different parameters.", 200

@app.route("/filter", methods=["GET"])
def filter_therapists():
    try:
        # parameter collection (from unity)
        qualifications = request.args.get("qualifications", "").strip()
        location = request.args.get("location", "").strip()
        availability = request.args.get("availability", "").strip()
        verification = request.args.get("verification", "").strip()
        min_endorsements = request.args.get("endorsements", "").strip()
        keyword = request.args.get("keyword", "").strip()
        limit = request.args.get("limit", "").strip()
        job_title = request.args.get("job_title", "").strip()
        accepting_clients = request.args.get("accepting_clients", "").strip()
        phone = request.args.get("phone", "").strip()

        # building sql WHERE clause
        base_query = """
            SELECT
                id,
                name,
                location,
                qualifications,
                job_title,
                availability,
                verification,
                endorsements,
                experience_years,
                sessions_completed,
                description,
```

```
            phone,
            accepting_clients
        FROM therapists
    """

    conditions = []
    values = []

    if qualifications:
        conditions.append("LOWER(qualifications) LIKE ?")
        values.append("%" + qualifications.lower() + "%")

    if location:
        conditions.append("LOWER(location) LIKE ?")
        values.append("%" + location.lower() + "%")

    if availability:
        conditions.append("LOWER(availability) LIKE ?")
        values.append("%" + availability.lower() + "%")

    if verification:
        # exact match (case-insensitive)
        conditions.append("LOWER(verification) = ?")
        values.append(verification.lower())

    if min_endorsements:
        try:
            min_val = int(min_endorsements)
            conditions.append("endorsements >= ?")
            values.append(min_val)
        except ValueError:
            return jsonify({"error": "endorsements must be an integer"}), 400

    if keyword:
        # match keyword in multiple text columns
        conditions.append("""(
            LOWER(description) LIKE ?
            OR LOWER(job_title) LIKE ?
            OR LOWER(qualifications) LIKE ?
        )""")
        kw = "%" + keyword.lower() + "%"
        values.extend([kw, kw, kw])

    if job_title:
        conditions.append("LOWER(job_title) LIKE ?")
        values.append("%" + job_title.lower() + "%")

    if accepting_clients:
        conditions.append("LOWER(accepting_clients) = ?")
        values.append(accepting_clients.lower())

    # (optional) WHERE combination
    if conditions:
```

```
            base_query += " WHERE " + " AND ".join(conditions)

        # (optional) adding limit
        if limit:
            try:
                limit_val = int(limit)
                base_query += " LIMIT ?"
                values.append(limit_val)
            except ValueError:
                return jsonify({"error": "limit must be an integer"}), 400

        # query execution
        conn = get_db_connection()
        cur = conn.cursor()
        cur.execute(base_query, values)
        rows = cur.fetchall()
        conn.close()

        # conversion into python dict
        results = []
        for r in rows:
            results.append({
                "ID": r["id"],
                "Name": r["name"],
                "Location": r["location"],
                "Qualifications": r["qualifications"],
                "Job_Title": r["job_title"],
                "Availability": r["availability"],
                "Verification": r["verification"],
                "Endorsements": r["endorsements"],
                "Experience_Years": r["experience_years"],
                "Sessions_Completed": r["sessions_completed"],
                "Description": r["description"],
                "Phone": r["phone"],
                "Accepting_Clients": r["accepting_clients"]
            })

        # error message if nothing: "No therapists found" (displayed in unity)
        return jsonify(results), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=True)
```

Appendix 10: Full HTTP_Test.cs script

```
using UnityEngine;
using UnityEngine.Networking;
using System.Collections;

public class HTTP_Test : MonoBehaviour
{
    void Start()
    {
        StartCoroutine(TestConnection());
    }

    IEnumerator TestConnection()
    {
        string url = "http://127.0.0.1:5000/"; // local host

        UnityWebRequest request = UnityWebRequest.Get(url);
        yield return request.SendWebRequest();

        if (request.result == UnityWebRequest.Result.Success)
        {
            Debug.Log("Connected! Response: " +
request.downloadHandler.text);
        }
        else
        {
            Debug.Log("Error: " + request.error);
        }
    }
}
```

Appendix 11: Full TherapistFilter.cs script

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Networking;
using UnityEngine.UI;
using TMPro;

[System.Serializable]
public class Therapist
{

    // defining all of the possible parameters
    public int ID;
    public string Name;
    public string Location;
    public string Qualifications;
    public string Verification;
    public string Endorsements;
    public string Availability;
    public int Experience_Years;
    public string Sessions_Completed;
    public string Description;
    public string Job_Title;
    public string Phone;
    public string Accepting_Clients;
}

[System.Serializable]
public class TherapistList
{
    public Therapist[] therapists;
}

public class TherapistFilter : MonoBehaviour
{

    [Header("Backend")]
    [SerializeField] private string backendUrl = "http://127.0.0.1:5000/filter";


    [Header("Input Fields")]
    public TMP_InputField locationInput;
    public TMP_InputField qualificationsInput;
    public TMP_InputField jobTitleInput;
    public TMP_InputField availabilityInput;
    public Toggle verificationToggle;
    public TMP_InputField endorsementsMinInput;
    public TMP_InputField limitInput;
    public TMP_InputField keywordInput;
    public TMP_InputField acceptingClientsInput;

    [Header("UI References")]
    public Button searchButton;
    public TextMeshProUGUI resultsText;
    public GameObject therapistCardPrefab;
```

```
public Transform resultsContainer;


// Start is called before the first frame update
void Start()
{
    if (searchButton != null)
    {
        searchButton.onClick.AddListener(SearchTherapists);
    }

    SearchTherapists(); // displays all therapists on start
}

public void SearchTherapists()
{
    StartCoroutine(GetFilteredTherapists());
}

// retrieves filtered therapists from given url
IEnumerator GetFilteredTherapists()
{
    resultsText.text = "Searching ...";

    string url = BuildFilterUrl();
    // Debug.Log("Requesting " + url);

    using (UnityWebRequest request = UnityWebRequest.Get(url))
    {
        yield return request.SendWebRequest();

        bool error = request.result != UnityWebRequest.Result.Success;

        if (error)
        {
            resultsText.text = "Error: " + request.error;
            yield break;
        }

        string jsonResponse = request.downloadHandler.text;

        DisplayResults(jsonResponse);

    }

}

private string BuildFilterUrl()
{
    string url = backendUrl + "?";
    Debug.Log("Requesting URL: " + url);

    bool firstParam = true;

    void AddParam(string key, string value)
    {
        if (!string.IsNullOrEmpty(value))
```

```
            {
                if (!firstParam)
                {
                    url += "&";
                }

                url += key + "=" + UnityWebRequest.EscapeURL(value);
                firstParam = false;
            }

        }

        void AddToggleParam(string key, Toggle toggle, string valueIfOn)
        {
            if (toggle != null && toggle.isOn)
            {
                AddParam(key, valueIfOn);
            }
        }

        AddParam("location", locationInput != null ? locationInput.text : "");
        AddParam("availability", availabilityInput != null ? availabilityInput.text : "");
        AddParam("qualifications", qualificationsInput != null ? qualificationsInput.text :
"");
        AddToggleParam("verification",  verificationToggle, "Verified");
        AddParam("endorsements", endorsementsMinInput != null ? endorsementsMinInput.text :
""); // min rating
        AddParam("limit", limitInput != null ? limitInput.text : "");
        AddParam("keyword", keywordInput != null ? keywordInput.text : "");
        AddParam("job_title", jobTitleInput != null ? jobTitleInput.text : "");
        AddParam("accepting_clients", acceptingClientsInput != null ?
acceptingClientsInput.text : "");

        return url;

    }

    private void DisplayResults(string jsonResponse)
    {

        // wrap raw array (from Flask) into an object so JsonUtility can parse it
        string wrappedJson = "{\"therapists\":" + jsonResponse + "}";

        TherapistList therapistList;

        try
        {
            therapistList = JsonUtility.FromJson<TherapistList>(wrappedJson);
            Debug.Log("Parsed therapists: " + (therapistList?.therapists == null ? "null" :
therapistList.therapists.Length.ToString()));
        }
        catch (System.Exception e)
        {
            Debug.LogError("JSON parse error: " + e.Message);
            resultsText.text = "Sorry, couldn't read results.";
            return;
        }
```

```
        // Clear previous cards
        foreach (Transform child in resultsContainer)
        {
            Destroy(child.gameObject);
        }

        // No matches edge case
        if (therapistList == null || therapistList.therapists == null ||
therapistList.therapists.Length == 0)
        {
            resultsText.text = "No therapists matched your filters.";
            return;
        }

        resultsText.text = therapistList.therapists.Length + " result(s)";

        // Make cards
        foreach (Therapist t in therapistList.therapists)
        {
            GameObject card = Instantiate(therapistCardPrefab, resultsContainer);

            // find text elements on the prefab
            TMP_Text nameField = card.transform.Find("NameText")?.GetComponent<TMP_Text>();
            TMP_Text qualField =
card.transform.Find("QualificationsText")?.GetComponent<TMP_Text>();
            TMP_Text locField =
card.transform.Find("LocationText")?.GetComponent<TMP_Text>();
            TMP_Text availField =
card.transform.Find("AvailabilityText")?.GetComponent<TMP_Text>();
            TMP_Text metaField = card.transform.Find("MetaText")?.GetComponent<TMP_Text>();
            TMP_Text jobTitleField =
card.transform.Find("JobTitleText")?.GetComponent<TMP_Text>();
            TMP_Text phoneField =
card.transform.Find("PhoneText")?.GetComponent<TMP_Text>();
            TMP_Text clientsField =
card.transform.Find("AcceptingClientsText")?.GetComponent<TMP_Text>();
            TMP_Text descField =
card.transform.Find("DescriptionText")?.GetComponent<TMP_Text>();
            TMP_Text experienceField =
card.transform.Find("ExperienceText")?.GetComponent<TMP_Text>();

            // fill them with clean labels (for aesthetics)
            if (nameField != null)
                nameField.text = SafeText(t.Name, "Name not listed");

            if (qualField != null)
                qualField.text = SafeText(t.Qualifications, "Qualifications not listed");

            if (locField != null)
                locField.text = SafeText(t.Location, "Location not listed");

            if (availField != null)
                availField.text = "Availability: " + SafeText(t.Availability, "Unknown");

            if (metaField != null)
            {
```

```
                // Build a compact summary line the client can scan fast
                string endorsePart = string.IsNullOrEmpty(t.Endorsements) ? "No rating" :
"Rating: " + t.Endorsements;
                string verifyPart = string.IsNullOrEmpty(t.Verification) ? "Unverified" :
t.Verification;
                metaField.text = endorsePart + "  •  " + verifyPart;

            }

            if (jobTitleField != null)
                jobTitleField.text = SafeText(t.Job_Title, "Job title not listed");

            if (phoneField != null)
                phoneField.text = SafeText(t.Phone, "Phone not listed");

            if (clientsField != null)
                clientsField.text = "Status: " + SafeText(t.Accepting_Clients, "Unknown");

            if (descField != null)
                descField.text = SafeText(t.Description, "No description available");

            if (experienceField != null)
                experienceField.text = "Experience: " + t.Experience_Years + " years";
        }
    }

    private string SafeText(string s, string fallback)
    {
        if (string.IsNullOrEmpty(s)) return fallback;
        return s;
    }

}
```

Appendix 12: Full FilterDropdownController.cs script

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using TMPro;

public class FilterDropdownController : MonoBehaviour
{
    [Header("Filter Toggles")]
    public Toggle locationToggle;
    public Toggle availabilityToggle;
    public Toggle qualificationsToggle;
    public Toggle verificationToggle;
    public Toggle endorsementsToggle;
    public Toggle keywordToggle;
    public Toggle limitToggle;
    public Toggle jobTitleToggle;
```

```
    public Toggle phoneToggle;
    public Toggle acceptingClientsToggle;

    [Header("Filter Groups (parents of label + input)")]
    public GameObject locationGroup;
    public GameObject availabilityGroup;
    public GameObject qualificationsGroup;
    public GameObject verificationGroup;
    public GameObject endorsementsGroup;
    public GameObject keywordGroup;
    public GameObject limitGroup;
    public GameObject jobTitleGroup;
    public GameObject phoneGroup;
    public GameObject acceptingClientsGroup;

    private void Start()
    {
        BindToggle(locationToggle, locationGroup);
        BindToggle(availabilityToggle, availabilityGroup);
        BindToggle(qualificationsToggle, qualificationsGroup);
        BindToggle(verificationToggle, verificationGroup);
        BindToggle(endorsementsToggle, endorsementsGroup);
        BindToggle(keywordToggle, keywordGroup);
        BindToggle(limitToggle, limitGroup);
        BindToggle(jobTitleToggle, jobTitleGroup);
        BindToggle(phoneToggle, phoneGroup);
        BindToggle(acceptingClientsToggle, acceptingClientsGroup);


        //SetAllGroupsActive(false);

    //     foreach (Toggle t in GetComponentsInChildren<Toggle>(true))
    // Debug.Log(t.name + " isOn: " + t.isOn);

    }

    private void BindToggle(Toggle toggle, GameObject group)
    {
        if (toggle == null || group == null) return;

        // make sure the group matches the toggle's initial state
        toggle.isOn = false;
        group.SetActive(false);

        GameObject capturedGroup = group;

        toggle.onValueChanged.AddListener((isOn) =>
        {
            if (isOn)
            SetAllGroupsActive(false); // all other groups hidden first

            capturedGroup.SetActive(isOn);
```

```
            // clear inputs when filter deactivated
            if (!isOn)
                ClearGroupInputs(capturedGroup);
        });

    }

    private void ClearGroupInputs(GameObject group)
    {

        foreach (var inputField in group.GetComponentsInChildren<TMP_InputField>(true))
            inputField.text = "";

    }

    private void SetAllGroupsActive(bool active)
    {
        SetGroupActive(locationGroup, active);
        SetGroupActive(availabilityGroup, active);
        SetGroupActive(qualificationsGroup, active);
        SetGroupActive(verificationGroup, active);
        SetGroupActive(endorsementsGroup, active);
        SetGroupActive(keywordGroup, active);
        SetGroupActive(limitGroup, active);
        SetGroupActive(jobTitleGroup, active);
        SetGroupActive(phoneGroup, active);
        SetGroupActive(acceptingClientsGroup, active);
    }

    private void SetGroupActive(GameObject group, bool active)
    {

        if (group != null)
            group.SetActive(active);

    }


}
```